



ICS_oct

Created @October 4, 2025 5:50 PM

写在前面：

本文档主要作为老师上课倡议的“考前关键知识梳理与总结”，希望能够结合PPT、书上的内容，做提炼与精简，形成体量不大但很精准的备考锦囊。

collaborated with Xisense wyx

下面列出的是写本文档过程中想到的应该覆盖的知识（更新）

信息的处理与表示，关于数在计算机中的存储、运算规则，应该会分

- ☐ int
- ☐ float,double
- ☐ implements

汇编，对我比较有挑战性的一章，几乎从零了解新的语言，目前感觉较需要记忆的是

- ☒ 1.寄存器及作用
- ☒ 2.指令集及语法
- ☐ 3.一些积攒的经验（？

汇编

第一部分：先从较多需要记忆、较新的汇编开始：

0 - 常用寄存器的作用

第1-4个参数：

%rdi,%rsi,%rdx,%rcx

5-6:

r8,r9

返回值：

%rax

栈指针：

%rsp

被调用者保存：

%rbx,%rbp,%r12-r15

低位访问：

32	16	8	
d	w	b	r8及之后
r改成e	去掉r	去掉r和x，末尾加l	...x
r改e	去r	去r,末尾+l	...p/i

第一个非常关键的操作是寻址：

3种形式：

直接（立即数Imm）：\$来代表格式，后面跟整型数，直接就访问了这个整型数

寄存器：%rax,etc. 这个直接就是访问了寄存器里面的值

而通过内存寻址的模式则最为复杂：

(1) Imm，即是常数，但没有\$，这种情况我们理解为访问这个常数地址中的内容

(2)(%r)，间接通过寄存器，这里我们找的是寄存器里面存的地址存的那个值

(2)Imm(rb,ri,s)：这是完整形态，访问Imm + rb + ri*s 这个地址存的值

下面我们重点回顾一些指令

1.mov S,D

后缀：

b 字节；w: 字；l：双字；q：四字

absq:绝对的四字

几种“传值”可能：

其实S没有限制，源可以是Imm，或者Reg,或者Mem

但是有两个限制：

D不能是立即数，必须是一个地址；

第二就是不能两个都是内存

所以一共只有5种情况

一定要注意的特例：

movl，且以寄存器作为目的时: 会把高4位置0

（也因此没有movzql）

而movabsq的用处就在于专门应对64位立即数数据的处理；因为movq只能处理32位的补码立即数，补符号位；而对于本身就64位的立即数，需要absq；但是注意：只能以寄存器作为D

只有一个字母后缀的指令自然要求S和D的长度都是和后缀指定长度相同；

而如果要把小赋给大，需要mov后面3个字母的后缀来指定：

z/s：

补0/符号位

紧接着倒数第二位指示S的大小，最后一位指示D的大小

C语言中的指针本质上就对应了“取地址”这个操作；

比如long *xp是参数，那么

%rdi → xp

(%rdi) → *xp

2.关于栈操作：本质上是栈指针sub/add的移动释放/回收了栈空间，然后执行的“mov”指令

栈顶指针%rsp

这里一定要注意的一个点是栈是“倒”的，即pushq对应栈指针地址的sub,而popq对应add

3.算术和逻辑运算相关指令

第一个很灵活也很好用的是leaq,注意这个指令也很特殊，没有变体，就是leaq

（其它都有不同后缀）

leaq虽然语法上是“加载有效地址，从内存读取数据到寄存器”，但实际上并没有访问“~~dest~~”s对应的内存，很快，常用于加法和有限形式乘法

其他的算术和逻辑指令都是有后缀的

区分一元和二元

一元包括

inc自增

dec自减

neg取负

not取反

二元

加减乘

异或XOR

或OR

与and

移位：（常用 $\times/\div 2$ 的幂）

shr是逻辑右移，sar是算术右移

sal,shl左移，但一样

特殊运算：8字（oct）

（完整的乘法可能溢出

称为“全乘法”：

只有一个操作数：

imulq有符号

mulq无符号

都是默认dest是%rax，另一个是源s

乘积会将高64位存在%rdx,低存在%rax

cqto：转化为8字，符号扩展

没有参数，就是把rax符号扩展到rdx

小端法机器：高位字节存在大地址（+8）

除法和取模操作：

单操作数：

idivl/q:被除数是%rdx + %rax固定，除数是由参数给出

将商存在%rax中，余数在%rdx

无符号divq:那么此时%rdx一般就设为0（没符号）

条件控制

0-基础

条件码

单个位，描述最近算术或逻辑操作属性

CF：结果最高位进位，检查无符号溢出

ZF：结果0

OF：溢出，+/-（有符号）

SF：符号标志，最近结果得到负数

无符号溢出判断标准就是被减数减完还变大了（肯定是0以下了）/加完变小了（截断了）

有符号则是符号相同的加/减爆了

应用条件码时注意：

leaq不会改变任何条件码，其他算术、逻辑运算指令会

移位操作：进位标志：最后一个被移出的位；溢出标志：0

inc,dec：设置溢出和0，但不改变进位

另外会设置条件码，但不改变寄存器的指令：

cmp,根据差来设置

test,根据“and”来设置，典型用法是取两个一样的操作数，看它是0还是正数/一个操作数选做掩码

条件码的读取与使用：

（1）set:(后缀是不同条件)，作用是满足条件时将唯一的参数即D一个字节设为1，否则为0

注意sets是负数条件，ns是非负

（对所有set都适用的设置条件码方式是cmp）

（2）条件跳转

rep空操作

用条件控制实现分支（传统的先判断，再分支内运算操作）

类比goto

（3）条件传送数据：先数据全算出来，然后用不同条件传送/选不同数据：受限可用，但现代处理器更快

- 循环实现

本质还是通过条件测试和跳转指令来实现

最基本do-while：结尾测条件

while：(1)jump to middle,先jump到结尾测试再看是否loop

(2)guarded-do:前面加一个初始测试之后改成do-while形式

for:可以转化成while(进一步也可以转成do-while)

switch：关键就是jump table

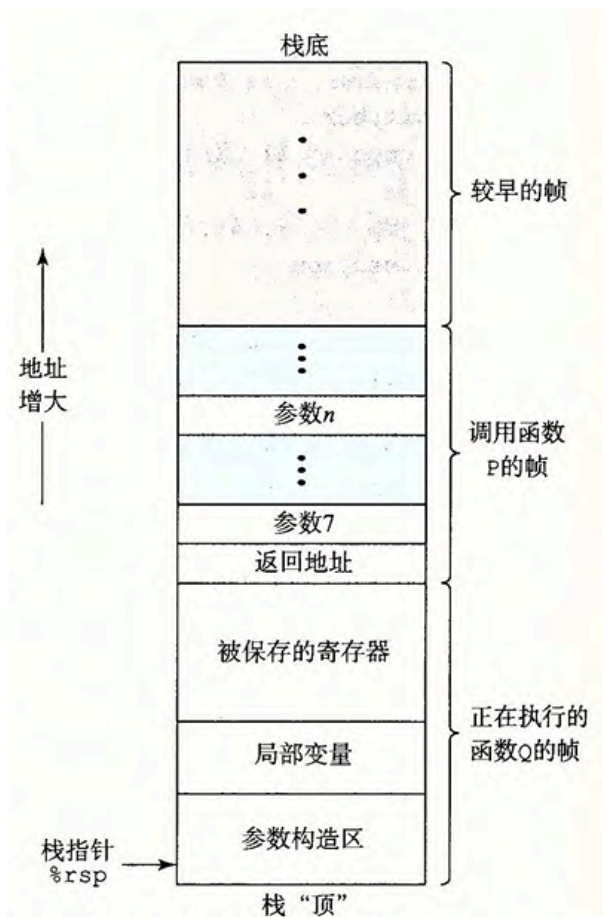
switch 语句在汇编中生成的跳转表（jump table）的顺序，与**case 常量的数值升序（从小到大）**顺序一致，而非源代码中 case 的书写顺序。

过程

0-basis

这一部分基础内容是运行时栈的情况

也即过程需要的存储空间超出寄存器能存放的大小时，在栈上分配空间，称为过程的栈帧



那么函数的调用和返回自然对应栈的push和pop

特别的，递归过程自然也适应于栈帧的架构，每个函数调用保有自己私有的状态信息存储空间，不会互相冲突

复杂数据结构实现

其实一言以蔽之，无非是复杂数据寻址模式的应用

数组、嵌套和高维数组

结构体、联合等异质数据结构

一个需要关注的是“数据对齐”

声明

.align 8

(空间换时间)

GDB及综合应用

先不赘述

信息表示和处理

首先，数的表示部分其实目前已经比较熟悉，

int就是用二进制、十六进制

float和double的规则须记忆

以及溢出须注意

具体见下：

整数表示与运算

无符号整数 vs 补码整数（w=32 位为例）

类型	表示规则	取值范围	示例（值 = 10）	示例（值 = -10）
无符号整数	所有位为数值位，值 = $\sum_{i=0}^{31} x_i \cdot 2^i$	$0 \sim 2^{32} - 1$ （约 42 亿）	0x0000000A	无（无法表示负）
补码整数	最高位为符号位，值 = $-x_{31} \cdot 2^{31} + \sum_{i=0}^{30} x_i \cdot 2^i$	$-2^{31} \sim 2^{31} - 1$ （约 ± 21 亿）	0x0000000A	0xFFFFFFFF6

整数溢出对比

类型	溢出定义	示例（w=4 位）	结果分析
无符号溢出	运算结果超出 $[0, 2^w-1]$	无符号 14 (1110) + 3 (0011) = 17	$17 \bmod 16 = 1$ (0001)，合法
有符号溢出	运算结果超出 $[-2^{w-1}, 2^{w-1}-1]$	补码 7 (0111) + 1 (0001) = 8	4 位补码最大为 7，溢出，结果为 - 8 (1000)，未定义行为

溢出风险：实际编程中需避免有符号溢出（如 `int a = 2147483647; a + 1` 会得到 - 2147483648，导致逻辑错误）。

移位运算对比（w=32 位，以 a=0x80000000 为例，补码表示 - 2^31）

移位类型	适用场景	操作规则	示例：a >> 1（右移 1 位）	示例：a << 1（左移 1 位）
逻辑移位	无符号整数	右移补 0，左移补 0	0x40000000（2^30）	0x00000000（溢出）
算术移位	补码整数（右移）	右移补符号位，左移补 0	0xC0000000（-2^30）	0x00000000（溢出）

浮点数表示与运算（IEEE 754 标准）

单精度（32 位）vs 双精度（64 位）结构

类型	总位数	符号位 S（1 位）	阶码 E（位数）	尾数 M（位数）	偏置值（B）	规格化值公式
单精度（float）	32	S（0 = 正，1 = 负）	8	23	127	$(-1)^S * (1+M) * 2^{E-B}$
双精度（double）	64	S（0 = 正，1 = 负）	11	52	1023	$(-1)^S * (1+M) * 2^{E-B}$

关键细节：尾数 M 隐含一个 “1”（规格化时），如 M=0.25（二进制 010...），实际值为 1.25，提升精度（23 位 M→24 位有效数字）。

浮点数分类与特殊值

分类	阶码 E（单精度）	尾数 M	表示值	示例（单精度）
规格化	$1 \leq E \leq 254$	任意	$(-1)^S * (1+M) * 2^{E-127}$	1.0 → S=0, E=127, M=0
非规格化	E=0	M≠0	$(-1)^S * M * 2^{-126}$	0.5×2^{-126} （接近 0）
零	E=0	M=0	$(-1)^S * 0$ （正零 / 负零）	0.0 → S=0, E=0, M=0
正无穷	E=255	M=0	$+\infty$	1.0 / 0.0
负无穷	E=255	M=0	$-\infty$	-1.0 / 0.0
NaN（非数）	E=255	M≠0	无效结果（如 0/0、sqrt(-1)）	0.0 / 0.0

浮点数运算特点

- 1. 精度丢失：由于尾数位数有限，部分十进制数无法精确表示（如 0.1 的二进制是无限循环小数，float/double 只能近似存储）。示例： $0.1 + 0.2 \neq 0.3$ （实际计算： $0.1 \approx 0.10000000149$ ， $0.2 \approx 0.20000000298$ ，和 $\approx 0.30000000447 \neq 0.3$ ）。
- 2. 不满足结合律： $(a + b) + c \neq a + (b + c)$ （因中间结果舍入）。
- 3. 舍入方式：默认 “向偶数舍入”（减少统计误差），其他方式包括向零、向正无穷、向负无穷。

内存中的数据组织

字节顺序对比（以 0x12345678 为例，地址从低到高）

字节顺序	定义	内存存储（低地址→高地址）	适用场景
大端（Big-endian）	高位字节存低地址	0x12 → 0x34 → 0x56 → 0x78	网络协议（TCP/IP）、文件格式（GIF/JPEG）
小端（Little-endian）	低位字节存低地址	0x78 → 0x56 → 0x34 → 0x12	x86/ARM（小端模式）、本地内存存储

运算部分实际上变化较多

重点注意舍入

考点总结

data部分

1. 进制转换：16和2的互转技巧 2和10的短除法
2. 大端小端序的问题
3. 逻辑运算(& & || !)和位运算(& ^ ~) 注意~的巧用和mask的使用
4. 整数表示（完全在吃屎）
5. unsigned和int之间的转换 特别是判断表达式中的转换关系
6. 零扩展（无符号数）和符号扩展（补码）
7. 整数截断公式：截断到k位就是 $\text{mod } 2^k$
8. 加法 判断溢出的方法
9. 乘法和除法 注意补码除法的向零取整的加bias过程($x + 2^{k-1}$)
10. IEEE754 浮点数规范标准 各个数的位长考试一定会考！float为18 23 double为11 52 分别对应符号 阶码 尾数
11. 浮点数的表示： $V = (-1)^s \times M \times 2^E$
 - a. 规格化的
 - i. exp不全为0或1的
 - ii. $E = e - bias; bias = 2^{k-1} - 1$ 偏置的大小和精度有关 float为127 double为1023 所以E的取值范围为-126~127 (-1022~1023)
 - iii. $M = 1.f_{n-1}f_{n-2} \cdots f_0$ 这里要求调整E的值使 $1 \leq M < 2$
 - b. 非规格化的
 - i. exp全为0
 - ii. $E = 1 - bias; M = 0.f_{n-1} \cdots f_0$
 - iii. +0 -0 两种不同编码的0可以被表示为非规格化数
 - iv. 主要用力啊表示非常接近0的数
 - c. 无穷大 exp全为1且frac部分全为0
 - d. NaN exp全为1且frac部分不为0
12. 浮点数运算真的会考吗 感觉好难啊

machine programming部分

怎么写，等会我看看

你看你吗呢？

考前可以再看看4 5 10

1. 基本概念

- a. 指令集构架 ISA 包括指令集规范和寄存器等
- b. 汇编代码
- c. 机器码

2. 由C向机器码的转变

- a. gcc -Og -S (生成汇编代码) -c (生成机器码) -o制定输出位置并连接生成可执行文件。
- b. 所以由C→可执行文件：C预处理器 (#include和#define)、编译器 (.s)、汇编器 (.o)、链接器 (目标代码育实现库函数合并，这一步需要有main函数)
- c. objdump -d xxx.o 用来进行反编译

3. 数据格式 注意浮点数的汇编表示不太一样 s和l单独记就好

C 声明	Intel 数据类型	汇编代码后缀 (记忆)	大小 (字节)
char	字节	b (byte)	1
short	字	w (word)	2
int	双字	l (double word)	4
long	四字	q (quad word)	8
char*	四字	q (quad word)	8
float	单精度	s (single word)	4
double	双精度	l (double word)	8

4. x86-64的基本体系结构

- a. PC程序计数器 是一个寄存器 (%rip) 内容指向下一个将要执行的指令地址，每次取指后就会更新
- b. 寄存器 Memory this figure
abcd、si、di、bp、sp r8-15 三种类型分别记还挺好记的

63	31	15	7	0	
%rax	%eax	%ax	%al		返回值
%rbx	%ebx	%bx	%bl		被调用者保存
%rcx	%ecx	%cx	%cl		第 4 个参数
%rdx	%edx	%dx	%dl		第 3 个参数
%rsi	%esi	%si	%sil		第 2 个参数
%rdi	%edi	%di	%dil		第 1 个参数
%rbp	%ebp	%bp	%bpl		被调用者保存
%rsp	%esp	%sp	%spl		栈指针
%r8	%r8d	%r8w	%r8b		第 5 个参数
%r9	%r9d	%r9w	%r9b		第 6 个参数
%r10	%r10d	%r10w	%r10b		调用者保存
%r11	%r11d	%r11w	%r11b		调用者保存
%r12	%r12d	%r12w	%r12b		被调用者保存
%r13	%r13d	%r13w	%r13b		被调用者保存
%r14	%r14d	%r14w	%r14b		被调用者保存
%r15	%r15d	%r15w	%r15b		被调用者保存

还要求知道其功能性：

%rax为返回值，

%rdi, %rsi, %rdx, %rcx, %r8, %r9为函数的前6个参数，

%rsp为栈指针，

%rbp, %rbx, %r12-15这六个寄存器为callee-saver（保存的义务由被调用的函数承担），即要求在执行中保存其中原有的值，也就说使用这些寄存器前请先pushq他们，除非你确定不会改变其中的值

%r10, %r11为调用者保存寄存器caller-saver（保存的义务由调用者承担），执行时可以随便改变值

5. 操作数指示符 注意64位系统的地址必须为64位，例如(%eax)不合法

a. 三种操作数类型：

i. 立即数 \$lmm 注意别忘了\$符号

ii. 寄存器 %xxx即可

iii. 内存引用 这个比较花

b. 内存引用寻址方法

- i. Imm : M[Imm] 绝对寻址, 注意不要和立即数搞混
- ii. Imm(r1,r2,s) M[Imm+r1+r2*s] 注意s=1, 2, 4, 8 这个结构省略一些部分也是对的

类型	格式	操作数值	名称
立即数	\$Imm	Imm	立即数寻址
寄存器	ra	R[ra]	寄存器寻址
存储器	Imm	M[Imm]	绝对寻址
存储器	(ra)	M[R[ra]]	间接寻址
存储器	Imm(rb)	M[Imm+R[rb]]	(基址 + 偏移量) 寻址
存储器	(rb, ri)	M[R[rb]+R[ri]]	变址寻址
存储器	Imm(rb, ri)	M[Imm+R[rb]+R[ri]]	变址寻址
存储器	(ri, s)	M[R[ri] * s]	比例变址寻址
存储器	Imm(ri, s)	M[Imm+R[ri] * s]	比例变址寻址
存储器	(rb, ri, s)	M[R[rb]+R[ri] * s]	比例变址寻址
存储器	Imm(rb, ri, s)	M[Imm+R[rb]+R[ri] * s]	比例变址寻址

6. mov类指令 mov s,d s→d

- a. movb、movw、movl、movq的区别 以及请注意一下易错点
 - i. mov指令的**两个操作数不能都是内存**, 且d操作数不能是立即数
 - ii. 操作数中出现的寄存器大小必须和mov后缀匹配
 - iii. movl以寄存器为目的时, 会将寄存器的高四位也设置为0
- b. movabsq Imm,R : 只能以立即数和寄存器作为操作数, 将64位数存在对应寄存器中
- c. movz型 (零扩展) : movzbw movzbl movzq movzwl movzwq 注意没有movzlw==movl
- d. movs (符号扩展) : movsbw movsbl movsbq movswl movswq movslq
- e. cltq指令 (无操作数) =movslq %eax,%rax 是将eax扩展位rax的指令 注意cltq只能作用于eax和rax上
- f. **特别注意** : x86-64中凡是以eax这种32位的寄存器作为回传操作数的计算, 寄存器高32位都会清零 movl、movsbl、addl、xorl等等

7. 栈 push和pop操作 注意顺序

- a. 栈从高地址向低地址延伸 (向下增长)
- b. 一般%rsp为栈顶位置, %rbp为栈底位置
- c. pushq S : sub \$8, %rsp; mov S, (%rsp) **先减后入栈**
- d. popq D : **先出栈后加**

8. 算术运算和逻辑运算 注意真正使用时也要加后缀bwlq

- a. lea S, D 加载有效地址（不引用内存）：其实主要用于运算寄存器中存储的数值，**注意D必须为寄存器，leaq不会改变条件码**
 - b. inc (+1)、dec (-1)、neg（取负）、not（取反）只有一个操作数，就保存在D中
 - c. add、sub、imul、xor、or、and 有S, D两个操作数，结果会保存在D中（即后面的操作数中）
 - d. sar、shr、sal、shl：h为逻辑，a为算术，l为左移，r为右移 写成xxx k,D：k是移位数，只能是Imm或%cl中的数 **注意%rcx可能因此出现截断**
9. 特殊的算术操作 一些八字运算，没考过的感觉 记住rdx在高位，绕行放结果就行
- a. imulq/mulq S：对应有符号数和无符号数 只有一个操作数S，计算为 $\%rdx:\%rax = S * \%rax$ 。使用前要先在rax中放好数值，结果为128位且rdx在高位
 - b. idivq/divq S：同样提前在rax中放好 S是除数， $rax = rax / S$ ， $rdx = rax \bmod S$
 - c. cqto：rax符号扩展为八字，同样rdx为高位
10. 标志位：
- a. 4个需要记住的标志位
 - i. ZF：运算结果为0则为1
 - ii. SF：运算结果为负则为1
 - iii. CF：运算结果无符号溢出则为1
 - iv. OF：运算结果有符号溢出则为1
 - b. 注意：
 - i. 计算机运算时并不区分有符号运算和无符号运算，所以无符号运算时出现了“符号溢出”，OF=1
 - ii. **leaq不会改变标志位**
 - iii. 逻辑运算会使CF、OF=0
 - iv. 移位操作：CF为最后一个被移出去的位，OF=0
 - v. inc和dec不会改变CF，但会设置OF和ZF
11. 条件码指令
- a. cmp：执行和sub相同的操作，但是不将结果保存（这里主要操作数顺序，一般会出现s2和s1进行比较）
 - b. test：执行&，同样不保存结果
 - c. set类指令获取标志位：目的操作数只能是一个字节的低位寄存器或一个字节内存，想要32位或64位需要movz一下
 - i. sete D：D=ZF或者说计算结果为0时置为1
 - ii. setne D：D= $\sim ZF$
 - iii. sets/setns D
 - iv. 有符号比较：setg setge setl setle
 1. 这里主要考察OF、ZF和SF，这里可以记住setg的表达式，其他都可以由其推出
 2. setg D 应该有SF和OF相同且ZF不为0时置1，所以给出 $D = \sim (SF \wedge OF) \wedge (\sim ZF)$
 3. setge $D = \sim (OF \wedge SF)$
 4. setl $D = OF \wedge SF$

5. $\text{setle } D = (\text{OF} \wedge \text{SF}) \mid \text{ZF}$

v. 无符号比较：seta setae setb setbe

1. 主要考察ZF、CF

2. seta D 这个更好考虑，只要越界就一定不是above了 $D = \sim \text{CF} \& (\sim \text{ZF})$

3. setae $D = \sim \text{CF}$

4. setb $D = \text{CF}$

5. setbe $D = \text{CF} \mid \text{ZF}$

d. 总之简单分析一下也是可以得到表达式的，不妨记住 $\text{setg } D = \sim (\text{OF} \wedge \text{SF}) \& (\sim \text{ZF})$ 和 $\text{seta} = \sim \text{CF} \& (\sim \text{ZF})$ 即可

12. 跳转指令 jmp

a. 直接跳转和间接跳转（`jmp *%rax`）即跳转到从%rax中读出的label上去，这里不需要再次取地址

b. 跳转指令编码时使用PC相对寻址，地址偏移可以为1，2，4字节，一般为1字节，注意地址偏移量是有符号的

13. 条件分支

a. 条件控制实现条件分支：通过jmp类条件控制类指令进行，后缀和set类相似

模版为：一般情况下都是先把不符合条件的转出去，然后直接执行符合条件的代码块，然后在向结尾跳转一步。

```
# C语言下的条件分支
if(test-expr){
    then-statement;
}
else{
    else-statement;
}

-----

# go-to版本
t=test-expr;
if(!t) goto false;
then-statement;
goto done;
false:
else-statement;
done:
```

b. 条件传送实现条件分支：cmov类指令，后缀同set，满足条件的时候就移动数据。不支持单字节b的传送，w/q由编译器识别决定，不需要显式地写出。

a. 原理是计算出所有分支下的值，判断后将满足要求的值移动到返回值上去

b. 只有两个表达式都比较简单，且不会出现错误时才会使用（例如空指针，地址运算和非法的算术）

14. 循环 狠狠的套模版

a. do-while

```
# C形式
```

```
do
    body-statement;
    while(test-expr)
```

goto形式

```
loop:
    body-statement;
    t=test-expr;
    if(t)goto loop;
```

b. while

C形式

```
while(test-expr)
    body-statement;
```

goto形式1 jump to middle 最开始的检查直接跳转到中间区域进行

```
    goto test;
loop:
    body-statement;
test:
    t=test-expr;
    if(t)goto loop;
```

goto形式2 guarded-do 单独进行检查，不成立就跳转到done，
这种方法就更偏向于初始条件基本是成立的

```
    t=test-expr;
    if(!t)goto done;
loop:
    body-statement;
    t=test-expr;
    if(t)goto loop;
done:
```

c. for

C

```
for(init-expr;test-expr;update-expr)
    body-statement;
```

translate to while

```
init-expr;
```

```
while(test-expr){
    body-statement;
    update-expr; //注意这里的先后顺序，先执行完主体后才更新值
}
```

goto版本

```
init-expr;
if(!test-expr)goto done;
loop:
    body-statement;
    update-statement;
    if(test-expr)goto loop;
done:
```

15. switch语句：

- 如果开关数量不多仍然会编译为条件分支
- 如果开关数量很多且出现了case比较连续的情况会使用jump table

16. 过程

- 机制
 - 传递控制：call和ret指令 PC中指令传递过程
 - 传递数据：P向Q传递一个或多个参数，Q向P传递一个返回值
 - 分配和释放空间：Q执行时需要为局部变量分配空间，执行结束后需要讲这些空间释放掉
- 实现是通过运行时栈的方式进行的 注意**栈帧**的概念 （也不是所有函数都需要栈帧）

x86-64 的栈结构

stack structure in x86-64

1. 栈帧布局：

- **参数构造区**：存放函数构造的参数
看图的上面，参数 7~n 就是 P 的参数构造区，注意顺序
- **局部变量区**：用于函数临时构造的局部变量。
- **被保存的寄存器**：用于保存调用过程中使用到的寄存器状态
被调用者保存寄存器： %rbp %rbx %r12 %r13 %r14 %r15
- **返回地址**：调用结束时的返回地址
返回地址属于调用者 P 的栈帧
- **对齐**：栈帧的地址必须是 16 的倍数
16 字节对齐，Attacklab 会用到哦

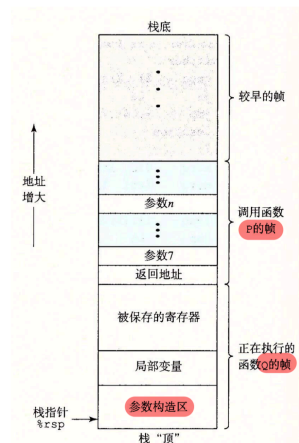


图 3-25 通用的栈帧结构(栈用来传递参数、存储返回信息、保存寄存器，以及局部存储。省略了不必要的部分)

c. 转移控制：在P中调用Q

- call Q：将下一条指令的地址压入栈中，将PC调整为Q的地址

1. call Label

2. call *(Operand)目标为寄存器或内存中的地址

ii. ret：弹出刚刚压入的地址，并将PC设置为该值

d. 数据传送：

i. 前六个参数：%rdi %rsi %rdx %rcx %r8 %r9

ii. 剩下的参数就需要使用栈传递了，第七个参数在栈顶（低地址），其他参数依此向后（向高地址排列）

iii. 注意其中的数据应该向8对齐，但是如果优化等级更高可以采取向k对齐的原则（不过书上似乎只说了向8对齐，那就向8对齐吧）

e. 栈上的局部存储：寄存器不够用 | 需要对某一个变量取地址 | 有数组等结构 才会使用（这里有个对齐的问题至今没搞明白）

f. 被保存的寄存器部分就保存了本函数callee-saver的内容

17. 数组的分配与访问

a. 指针运算：指针的加减会自动乘数据类型的步长

b. T A[N]

i. 数据类型T的大小为L

ii. 内存中分配L*N的连续空间

iii. 访问公式： $A[i] = M[x_A + Li]$

c. 二维数组 T A[R][C] 访问公式为 $A[i][j] = M[x_A + L(Ci + j)]$

d. 复杂表达式解码 int* (*p[2])[3] 反复读就完事了，遇到 () 还可以读成函数，返回值为...

e. 使用sizeof运算（考试必考，但是也没啥好办法，总感觉模棱两可的）

18. 数据结构

a. struct和union两种数据结构，union所有数据公用一个空间，所以分配时分配最大的即可，struct要遵循任何k字节的基本对象的地址必须是k的倍数的原则。

在结构体的内存中，有一部分为了对齐而空出的字节

- 内部填充：为了满足每个长度为 K 的数据相对于首地址的偏移都是 K 的倍数
- 外部填充：为了满足内存总长度是最大的 K 的倍数

b. 嵌套struct和union的结构体对齐：嵌套部分的大小和align需要分开来看

c. 还有一些特例：

i. 栈帧必须向16对齐

ii. 参数构造区向8对齐

iii. 内存分配函数起始地址也是16对齐的

19. 最后一部分是第八讲的内容，主要是栈等内容，看完再说，可能也不想写了，就这吧

整理一点点往年题

1.数据对齐：

基本原则是k字节的基本对象的地址必须是k的倍数

2.左右法则

在 C 语言中，“左右法则”是解析复杂声明（尤其是包含指针、数组、函数的混合声明）的常用方法，其核心思路是：从变量名出发，先看右边，再看左边，遇到括号先处理括号内的部分，逐步确定变量的类型。

左右法则的具体步骤：

1. **找到变量名**：从声明中定位到要解析的变量名（如题目中的 `p`）。
2. **先右后左**：从变量名开始，优先查看右边的符号（如 `[]` 表示数组，`()` 表示函数），再查看左边的符号（如表示指针）。
3. **处理括号**：若遇到括号 `()`，先解析括号内的部分（括号会改变优先级）。
4. **逐层扩展**：重复上述步骤，逐步明确变量的完整类型（是数组、指针，还是指向数组的指针、数组元素为指针等）。

用左右法则解析题目中的声明：`struct student* (*p[8])[4];`

我们以变量名 `p` 为起点，逐步解析：

1. **定位变量名**：声明的核心是 `p`，需要确定 `p` 是什么。
2. **看 `p` 的右边**：`p` 右边是 `[8]`，表示 `p` 是一个数组，且数组有 8 个元素。
3. **看括号内的左边**：`p` 被括号 `(*p[8])` 包围，括号内左边是，表示数组 `p` 的每个元素是指针。
4. **解析指针指向的类型**：跳出括号后，看这个指针的右边是 `[4]`，表示该指针指向一个数组，且这个被指向的数组有 4 个元素。
5. **解析被指向数组的元素类型**：最后看最左边的 `struct student*`，表示被指向的数组的每个元素是指向 `struct student` 的指针。

最终解析结果：

`p` 是一个包含 8 个元素的数组，数组的每个元素是指针，这些指针指向一个包含 4 个元素的数组，而这个被指向的数组的每个元素是指向 `struct student` 的指针。

通过左右法则，我们能清晰拆解复杂声明的层次，这也是理解题目中 `sizeof(**p)` 等计算的基础。

3. 由于栈是从高地址向低地址生长（`push` 操作会使栈指针 `%rsp` 递减），假设存在参数 `arg7`、`arg8`、`arg9` 需要通过栈传递，压栈顺序是先压 `arg9`，再压 `arg8`，最后压 `arg7`（`arg9` 在高地址，`arg7` 在低地址）。因此，“参数 7 最先压栈”的说法是错误的，选项 C 为错误选项。
- 4.

错题整理：

1.关于unsigned和int的比较/运算

默认转化成unsigned,注意是逐位转化,而不是负变正:

比如

-10 > 10U

2.关于截断与扩展:

截断就是直接低位截断;

关于负数:我们可以用一种很好的方法推表示:

先用最短的数位来表示,当然首位一定是1,之后我们只需在前面一直扩展1即可

int转double之后一定是准确值,所以此时加法有结合律;

但乘法没有,因为有可能溢出!!

主要是精度问题,乘法32*32最大可能63位,但double的尾数只能保留32位

3.访问 (%rax)也算要访问一次寄存器!,当然也算访问一次内存

4.test是and!!!不是add

5.l是对应双字,4个字节,

字是w(word)不要记混

6.有些情况不适合使用条件传送指令,因为条件传送指令需要:把条件成立和不成立的两种情况都提前计算出来,会增加计算量,还有可能有副作用:

比如空指针(副作用),以及两个分支对同一个数做修改(++)那么会有问题

7. for循环!

在 C 语言的 `for` 循环中,执行顺序为:先执行循环体内部的代码,再执行括号内的 `mask` 更新操作。

`for` 循环的执行流程是:

1. 执行初始化 (`mask = 1` , 仅执行一次);
2. 判断条件 (`mask != 0`), 若满足则进入循环体;
3. 执行循环体 (`result |= x&mask;`);
4. 执行迭代更新 (`mask = mask<<n`);
5. 回到步骤 2, 重复条件判断→循环体→迭代更新的流程,直到条件不满足。

因此，是循环体内的代码先运行，之后才执行括号内的 `mask` 更新。

8. 栈帧！调用函数的逻辑

首先在执行这个call这一条指令之后，主要操作有两个：

1. `subq $8, %rsp` 栈指针还是在这个caller的栈帧中，我们要给callee的返回值分配空间、然后我们注意，`%rsp` 指向的内存中的数值为目前这一条call指令的数值，保留待会ret的位置
2. `rip`指向callee的内存

然后才会进入callee,callee中也未必再会产生栈帧，要看它的参数、返回值是否在寄存器中

9. 注意指令、寄存器长度！

`imulq!!`

注意：汇编代码要从上往下一点点看，不能跳着看！！

甚至还有错题整理

卷起来了qwq